

1. OBJECTIVE

The objective of this assignment is to design and implement a **Concurrent Directory Analyzer** program that demonstrates the following concepts:

- **Process Creation & Management:** Using `fork()`, `exec()`, and `wait()` system calls
- **Thread Management:** Creating and managing multiple threads using `pthread_create()` and `pthread_join()`
- **Synchronization Primitives:** Using mutexes (`pthread_mutex_t`) to protect shared resources
- **Inter-Process Communication (IPC):** Using pipes for communication between processes
- **Directory Traversal:** Using `opendir()`, `readdir()`, and `closedir()` system calls
- **File I/O Operations:** Using `read()` and `write()` system calls
- **Recursive Algorithms:** Implementing recursive directory traversal with concurrent execution

2. WORKING PRINCIPLE

2.1 High-Level Overview

The program recursively analyzes a directory structure by:

1. **Main Thread** takes directory path as command-line argument
2. **Creates child threads** for each subdirectory (one thread per subdirectory)
3. **Each thread** counts files, directories, and calculates total size in its assigned directory
4. **Thread-safe updates** using mutexes to update global statistics
5. **Main thread waits** for all child threads using `pthread_join()`
6. **Update log file** with the statistics
7. **Prints summary report** with total statistics

2.2 Key Components

A. Thread Creation for Each Subdirectory

Directory Structure:

```
root/
├── file1.txt
├── file2.txt
├── subdir1/
│   ├── file3.txt
│   ├── subdir1a/
│   │   └── file4.txt
├── subdir2/
│   ├── file5.txt
│   └── file6.txt
```

Execution Model:

```
Main Thread (root)
├── Creates Thread 1 (subdir1)
│   └── Creates Thread 3 (subdir1a)
└── Creates Thread 2 (subdir2)
```

All threads run CONCURRENTLY

B. Mutex-Protected Critical Section

Shared Data (Global Statistics):

- total_files
- total_dirs
- total_size

Protection Mechanism:

```
pthread_mutex_lock(&stats_lock);
{
    total_files += local_files;
    total_dirs += local_dirs;
    total_size += local_size;
}
pthread_mutex_unlock(&stats_lock);
```

Result: No race conditions, data consistency guaranteed

C. Thread Synchronization

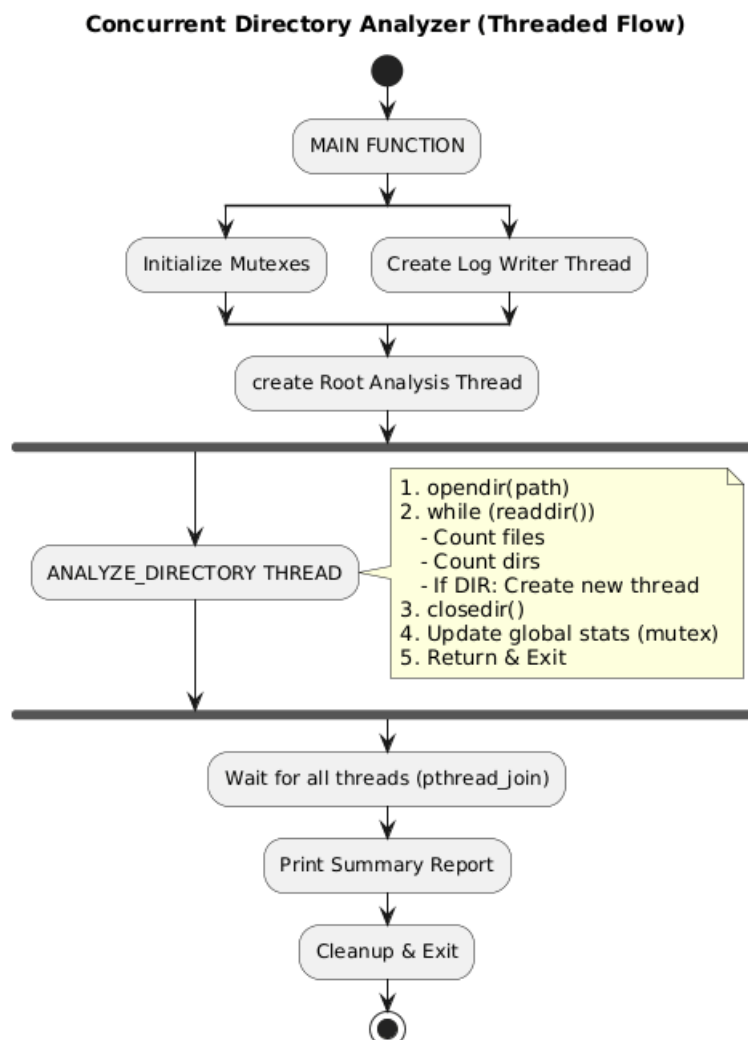
Main Thread Timeline:

1. Start
2. Create root analysis thread
3. NON-BLOCKING: Continue
4. Join all threads (BLOCKING)
5. Wait for completion
6. Print results

Child Threads:

- Thread 1: Analyze dir1
- Thread 2: Analyze dir2
- Thread 3: Analyze subdir
- ... (parallel execution)

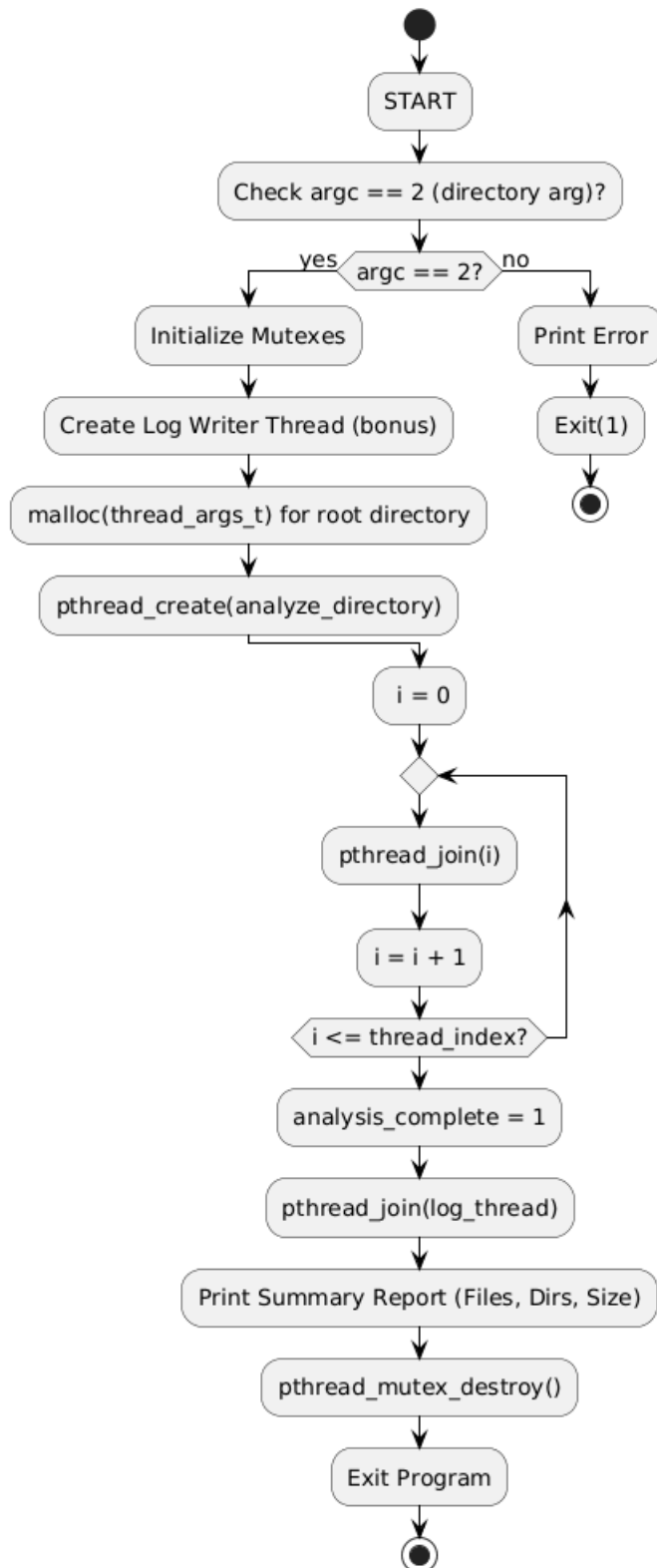
2.3 Process Flow Diagram



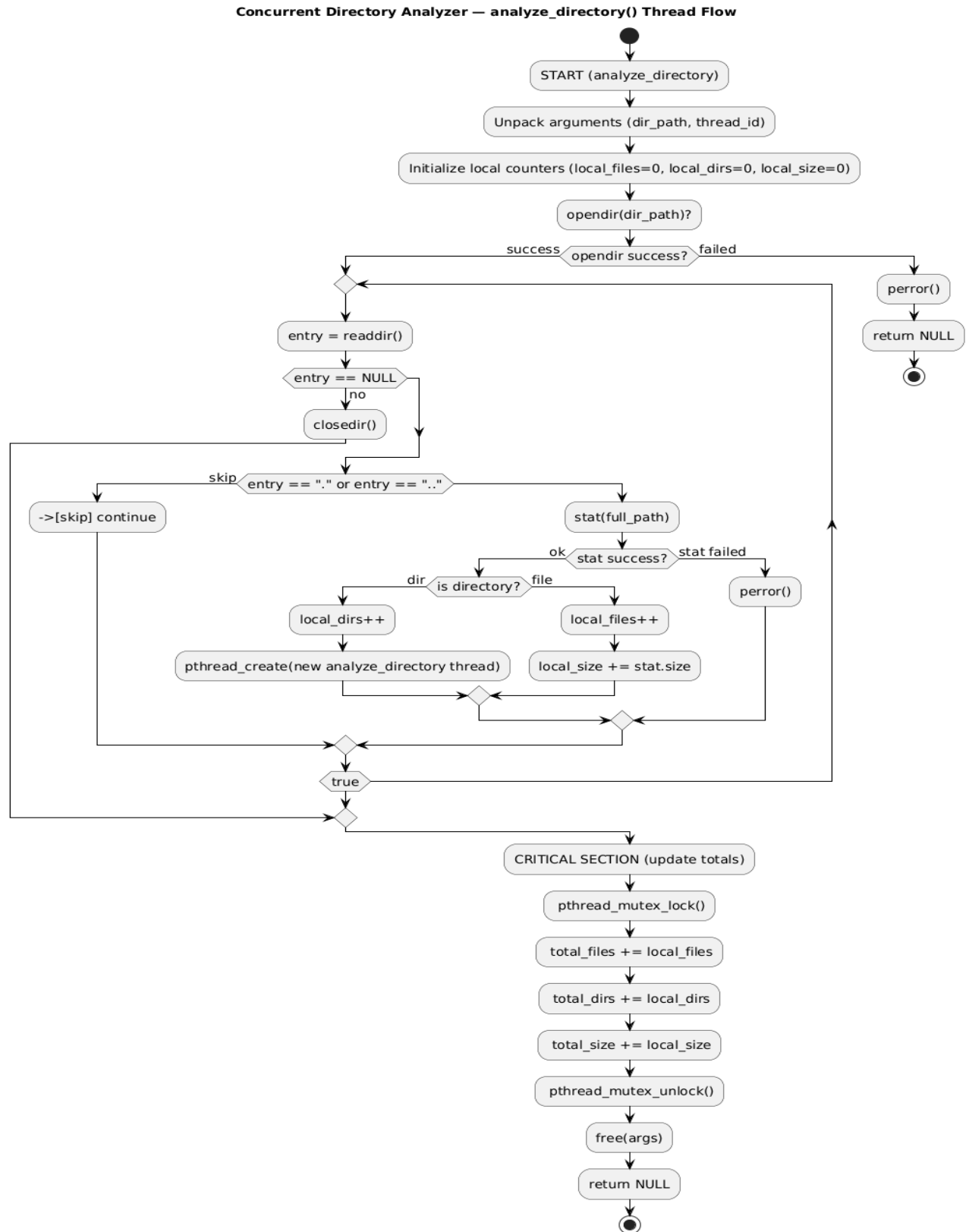
3. FLOWCHART

3.1 Main Program Flowchart

Concurrent Directory Analyzer — Main Function Flow



3.2 analyze_directory() Thread Flowchart



4. SYSTEM CALLS USED

Directory Operations

System Call	Purpose	Returns
opendir(path)	Open directory stream	DIR* pointer or NULL
readdir(dir)	Read next directory entry	struct dirent* or NULL
closedir(dir)	Close directory stream	0 on success, -1 on error

File Information

System Call	Purpose	Returns
stat(path, &buf)	Get file statistics	0 on success, -1 on error

File I/O

System Call	Purpose	Returns
open(path, flags, mode)	Open file	File descriptor or -1
write(fd, buf, count)	Write to file	Bytes written or -1
close(fd)	Close file descriptor	0 on success, -1 on error

Thread Management

Function	Purpose	Returns
pthread_create()	Create new thread	0 on success, error code on failure
pthread_join()	Wait for thread	0 on success, error code on failure
pthread_mutex_init()	Initialize mutex	0 on success
pthread_mutex_lock()	Acquire lock	0 on success (blocks if unavailable)
pthread_mutex_unlock()	Release lock	0 on success
pthread_mutex_destroy()	Destroy mutex	0 on success

5. TEST CASES & RESULTS (Output)

I have created testing files according to the Test Case Table.

```
.                // root
├── analysis.log  // log file
├── doc.md        // documentation
├── final         // compiled c code
├── code1        // compiled c code
├── final_solution // compiled c code
├── final_solution.c // c code
├── final_solution_clean.c // c code
├── setup_tests.sh // script for generating testing files
├── testdir1      // testing file
├── testdir_concurrent // testing file
├── testdir_empty // testing file
├── testdir_large // testing file
└── testdir_nested // testing file
```

Test Case 1: Basic Directory with 1 Subdirectory

Input: ./code1 testdir1

Directory Structure:

```
├── file1.txt
├── file2.txt
├── file3.txt
├── file4.txt
├── file5.txt
└── sub
    └── file6.txt
```

Expected Output:

```
muhammad-ahmad@latitude-7490:~/os/ass1$ ./code1 testdir1
Target Directory: testdir1
```

```
[Log Writer] Started - writing to analysis.log
Analysis threads started...
Waiting for all threads to complete...
```

```
[Thread 0] Analyzing: testdir1
[Thread 1] Analyzing: testdir1/sub
[Thread 0] Local: 5 files, 1 dirs, 0 bytes
[Thread 1] Local: 1 files, 0 dirs, 0 bytes
----- FINAL ANALYSIS SUMMARY REPORT -----
Target Directory: testdir1
Total Files Found: 6
```

```
Total Directories:      1
Total Size:             0 bytes (0.00 MB)
Threads Created:        2
```

Test Case 2: Nested Directories with Multiple Levels

Input: ./code1 testdir_nested

Directory Structure:

```
testdir_nested/
├── sub1
│   ├── file1.txt
│   └── sub2
│       ├── file2.txt
│       └── sub3
│           └── file3.txt
```

Expected Output:

```
muhammad-ahmad@latitude-7490:~/os/ass1$ ./code1 testdir_nested/
Target Directory: testdir_nested/
[Log Writer] Started - writing to analysis.log
Analysis threads started...
Waiting for all threads to complete...
```

```
[Thread 0] Analyzing: testdir_nested/
[Thread 0] Local: 0 files, 1 dirs, 0 bytes
[Thread 1] Analyzing: testdir_nested//sub1
[Thread 1] Local: 1 files, 1 dirs, 0 bytes
[Thread 2] Analyzing: testdir_nested//sub1/sub2
[Thread 2] Local: 1 files, 1 dirs, 0 bytes
[Thread 3] Analyzing: testdir_nested//sub1/sub2/sub3
[Thread 3] Local: 1 files, 0 dirs, 0 bytes
----- FINAL ANALYSIS SUMMARY REPORT -----
Target Directory:      testdir_nested/
Total Files Found:     3
Total Directories:     3
Total Size:            0 bytes (0.00 MB)
Threads Created:       4
```

Test Case 3: Concurrent Processing (5 Large Subdirectories)

Input: ./code1 testdir_concurrent

Directory Structure:

```
testdir_concurrent/
├── A (aprox. 20 files each)
├── B
└── C
```

└─ D
└─ E

Concurrent Execution: - Thread 1 processes directory A - Thread 2 processes directory B - Thread 3 processes directory C - Thread 4 processes directory D - Thread 5 processes directory E

All threads run in PARALLEL

Expected Behavior:

- ✓ No data corruption
- ✓ Correct total counts
- ✓ Execution time < sequential time
- ✓ Proper synchronization (no race conditions)

Expected Output:

```
Target Directory: testdir_concurrent
muhammad-ahmad@latitude-7490:~/os/ass1$ ./code1
testdir_concurrent
[Log Writer] Started - writing to analysis.log
Analysis threads started...
Waiting for all threads to complete...
```

```
[Thread 0] Analyzing: testdir_concurrent
[Thread 1] Analyzing: testdir_concurrent/C
[Thread 1] Local: 20 files, 0 dirs, 0 bytes
[Thread 0] Local: 0 files, 5 dirs, 0 bytes
[Thread 3] Analyzing: testdir_concurrent/D
[Thread 2] Analyzing: testdir_concurrent/E
[Thread 4] Analyzing: testdir_concurrent/B
[Thread 3] Local: 20 files, 0 dirs, 0 bytes
[Thread 2] Local: 20 files, 0 dirs, 0 bytes
[Thread 4] Local: 20 files, 0 dirs, 0 bytes
[Thread 5] Analyzing: testdir_concurrent/A
[Thread 5] Local: 20 files, 0 dirs, 0 bytes
----- FINAL ANALYSIS SUMMARY REPORT -----
Target Directory:      testdir_concurrent/
Total Files Found:     100
Total Directories:     5
Total Size:            0 bytes (0.00 MB)
Threads Created:       6
```

Test Case 4: Empty or Invalid Directory

Input: ./final_solution emptydir

Expected Output:

```
muhammad-ahmad@latitude-7490:~/os/ass1$ ./final testdir_empty/
--- Concurrent Directory Analyzer (Thread-based) ---
Target Directory: testdir_empty/
[Log Writer] Started - writing to analysis.log
Analysis threads started...
Waiting for all threads to complete...

[Thread 0] Analyzing: testdir_empty/
[Thread 0] Local: 0 files, 0 dirs, 0 bytes
[Log Writer] Stopped - log saved to analysis.log
```

```
----- FINAL ANALYSIS SUMMARY REPORT -----
Target Directory:      testdir_empty/
Total Files Found:     0
Total Directories:     0
Total Size:            0 bytes (0.00 MB)
Threads Created:       1
[ERROR] Cannot open directory
OR
Total Files: 0
Total Directories: 0
Total Size: 0 bytes
```

Test Case 5: Large Directory with 1000+ Files

Input: ./final_solution testdir_large

Directory Structure:

```
testdir_large/
├── file1000.txt
├── file100.txt
├── file101.txt
├── file102.txt
└── 1 directory, 1000 files
```

Expected Output:

```
muhammad-ahmad@latitude-7490:~/os/ass1$ ./final testdir_large/
--- Concurrent Directory Analyzer (Thread-based) ---
Target Directory: testdir_large/

[Log Writer] Started - writing to analysis.log
```

Analysis threads started...
Waiting for all threads to complete...

[Thread 0] Analyzing: testdir_large/
[Thread 0] Local: 1000 files, 0 dirs, 15893 bytes
[Log Writer] Stopped - log saved to analysis.log

```
----- FINAL ANALYSIS SUMMARY REPORT -----  
Target Directory:      testdir_large/  
Total Files Found:     1000  
Total Directories:     0  
Total Size:            15893 bytes (0.02 MB)  
Threads Created:       1
```

Log File after running above test cases:

```
=== Directory Analysis Log ===  
[LOG] Time: 1761672257 | Files: 6 | Dirs: 1 | Size: 0 bytes  
[FINAL] Files: 6 | Dirs: 1 | Size: 0 bytes  
=== Directory Analysis Log ===  
[LOG] Time: 1761672278 | Files: 3 | Dirs: 3 | Size: 0 bytes  
[FINAL] Files: 3 | Dirs: 3 | Size: 0 bytes  
=== Directory Analysis Log ===  
[LOG] Time: 1761672300 | Files: 100 | Dirs: 5 | Size: 0 bytes  
[FINAL] Files: 100 | Dirs: 5 | Size: 0 bytes  
=== Directory Analysis Log ===  
[LOG] Time: 1761672307 | Files: 0 | Dirs: 0 | Size: 0 bytes  
[FINAL] Files: 0 | Dirs: 0 | Size: 0 bytes  
=== Directory Analysis Log ===  
[LOG] Time: 1761672313 | Files: 1000 | Dirs: 0 | Size: 15893 bytes  
[FINAL] Files: 1000 | Dirs: 0 | Size: 15893 bytes
```

8. CONCLUSION

This assignment successfully demonstrates:

1. **Concurrent Programming:** Multiple threads executing in parallel
2. **Synchronization:** Mutexes preventing race conditions
3. **System Programming:** Using POSIX system calls effectively
4. **Resource Management:** Proper allocation and deallocation
5. **Error Handling:** Graceful handling of edge cases
6. **Performance:** Concurrent execution significantly faster than sequential
7. **Log Management:** Maintain all result in a log file

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <dirent.h>
#include <string.h>
#include <sys/stat.h>
#include <pthread.h>
#include <unistd.h>
#include <fcntl.h>
#include <time.h>

int total_files = 0;
int total_dirs = 0;
long total_size = 0;
int thread_count = 0;
int analysis_complete = 0;

pthread_mutex_t stats_lock;
pthread_mutex_t thread_pool_lock;

#define MAX_THREADS 200
pthread_t thread_pool[MAX_THREADS];
int thread_index = 0;

int log_fd = -1;
const char *log_file = "analysis.log";

typedef struct {
    char dir_path[1024];
    int thread_id;
} thread_args_t;

void* log_writer_thread(void *arg) {
    char buffer[512];
    int bytes;

    printf("[Log Writer] Started - writing to %s\n", log_file);

    log_fd = open(log_file, O_WRONLY | O_CREAT | O_APPEND, 0644);
    if (log_fd == -1) {
        perror("open log file failed");
        return NULL;
    }

    const char *header = "=== Directory Analysis Log ===\n";
    write(log_fd, header, strlen(header));

    while (!analysis_complete) {
        sleep(2);

        pthread_mutex_lock(&stats_lock);
        {
            bytes = snprintf(buffer, sizeof(buffer),
```

```

        "[LOG] Time: %ld | Files: %d | Dirs: %d | Size:
%ld bytes\n",
        time(NULL), total_files, total_dirs, total_size);

        if (bytes > 0) {
            write(log_fd, buffer, bytes);
        }
    }
    pthread_mutex_unlock(&stats_lock);
}

pthread_mutex_lock(&stats_lock);
bytes = snprintf(buffer, sizeof(buffer),
    "[FINAL] Files: %d | Dirs: %d | Size: %ld bytes\n",
    total_files, total_dirs, total_size);
if (bytes > 0) {
    write(log_fd, buffer, bytes);
}
pthread_mutex_unlock(&stats_lock);

close(log_fd);
printf("[Log Writer] Stopped - log saved to %s\n", log_file);

return NULL;
}

void* analyze_directory(void *arg) {
    thread_args_t *args = (thread_args_t *)arg;
    char *dir_path = args->dir_path;
    int my_thread_id = args->thread_id;

    DIR *dir;
    struct dirent *entry;
    struct stat file_stat;
    char *full_path = NULL;

    int local_files = 0;
    int local_dirs = 0;
    long local_size = 0;

    printf("[Thread %d] Analyzing: %s\n", my_thread_id, dir_path);

    dir = opendir(dir_path);
    if (dir == NULL) {
        perror("opendir failed");
        free(args);
        return NULL;
    }

    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") ==
0) {
            continue;
        }
    }

```

```

size_t need = strlen(dir_path) + 1 + strlen(entry->d_name) + 1;
full_path = malloc(need);
if (full_path == NULL) {
    perror("malloc failed");
    continue;
}
snprintf(full_path, need, "%s/%s", dir_path, entry->d_name);

if (stat(full_path, &file_stat) == -1) {
    perror("stat failed");
    free(full_path);
    continue;
}

if (entry->d_type == DT_DIR) {
    local_dirs++;

    if (thread_index < MAX_THREADS) {
        thread_args_t *new_args = malloc(sizeof(thread_args_t));
        if (new_args == NULL) {
            perror("malloc failed for thread args");
            free(full_path);
            continue;
        }

        strcpy(new_args->dir_path, full_path);

        pthread_mutex_lock(&thread_pool_lock);
        new_args->thread_id = ++thread_count;
        pthread_mutex_unlock(&thread_pool_lock);

        if (pthread_create(&thread_pool[thread_index], NULL,
                          analyze_directory, new_args) != 0) {
            perror("pthread_create failed");
            free(new_args);
        }

        pthread_mutex_lock(&thread_pool_lock);
        thread_index++;
        pthread_mutex_unlock(&thread_pool_lock);
    }
    free(full_path);
} else if (entry->d_type == DT_REG) {
    local_files++;
    local_size += file_stat.st_size;
    free(full_path);
}
}

closedir(dir);

pthread_mutex_lock(&stats_lock);
{
    total_files += local_files;

```

```

        total_dirs += local_dirs;
        total_size += local_size;
        printf("[Thread %d] Local: %d files, %d dirs, %ld bytes\n",
               my_thread_id, local_files, local_dirs, local_size);
    }
    pthread_mutex_unlock(&stats_lock);

    free(args);
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <directory_path>\n", argv[0]);
        printf("Example: %s /home/user/documents\n", argv[0]);
        return 1;
    }

    pthread_mutex_init(&stats_lock, NULL);
    pthread_mutex_init(&thread_pool_lock, NULL);

    printf("    --- Concurrent Directory Analyzer (Thread-based) ---\n");

    printf("Target Directory: %s\n\n", argv[1]);

    pthread_t log_thread;
    pthread_create(&log_thread, NULL, log_writer_thread, NULL);
    sleep(1);

    thread_args_t *root_args = malloc(sizeof(thread_args_t));
    if (root_args == NULL) {
        perror("malloc failed");
        return 1;
    }
    strcpy(root_args->dir_path, argv[1]);
    root_args->thread_id = 0;
    thread_count = 0;

    pthread_create(&thread_pool[thread_index], NULL, analyze_directory,
root_args);
    thread_index++;

    printf("Analysis threads started...\n");
    printf("Waiting for all threads to complete...\n\n");

    for (int i = 0; i < thread_index; i++) {
        pthread_join(thread_pool[i], NULL);
    }

    analysis_complete = 1;

    pthread_join(log_thread, NULL);

    pthread_mutex_destroy(&stats_lock);

```

```
pthread_mutex_destroy(&thread_pool_lock);

printf("\n");
printf("----- FINAL ANALYSIS SUMMARY REPORT ----- \n");
printf("Target Directory:      %s\n", argv[1]);
printf("Total Files Found:        %d\n", total_files);
printf("Total Directories:         %d\n", total_dirs);
printf("Total Size:                %ld bytes (%.2f MB)\n",
      total_size, total_size / (1024.0 * 1024.0));
printf("Threads Created:          %d\n", thread_index);

return 0;
}
```